Nim

Продвинутые возможности и внутреннее устройство

Память, метапрограммирование и выполнение задач

Житков Никита · Сагун Александр · Жуков Александр

ARC / ORC VM компилятора Парадигмы Async / Await Потоки

Управление памятью: ARC и ORC

ARC

Автоматический подсчёт ссылок.
Освобождение при исчезновении последней владеющей ссылки.

ORC

Подсчёт ссылок со сборкой циклов.
Стандартный режим Nim 2.x.
Используется в нашем проекте.

Циклические ссылки

Два объекта могут удерживать друг друга, даже когда программа больше не имеет к ним доступа.



ARC не собирает такие циклы.
ORC обнаруживает их и освобождает.

Деструктор и время жизни объекта

p01_memory.nim

```
type MyResource = object
  id: int

proc `=destroy`(x: MyResource) =
  echo "Уничтожен ресурс с id=", x.id

block:
  var res = MyResource(id: 42)
  echo "Создан ресурс с id=", res.id

echo "Блок завершен, ресурс уничтожен."
```

ПОРЯДОК СООБЩЕНИЙ

```
Создан ресурс с id=42
Уничтожен ресурс с id=42
```

Затем появляется строка о завершении блока.

=destroy вызывается автоматически для локального res при выходе из блока.

Деструкторы управляют ресурсами. Перемещения позволяют сокращать копирование.

Вычисления в VM компилятора

p02_vm.nim · ФРАГМЕНТ

```
proc compileTimeCalc(): int =
  var sum = 0
  for i in 1..10:
    sum += i
  result = sum

static:
  let value = compileTimeCalc()
  echo "Результат вычислений в VM: ", value
```

CTFE

Выполнение функций при компиляции.

55

Сумма чисел от 1 до 10. Вычисляется до запуска.

static требует выполнения на этапе компиляции. Доступна поддерживаемая VM часть Nim.

Компиляция и запуск программы

ШАГ 1 · КОМПИЛЯЦИЯ

```
nim c -f main.nim
```

Выполняется static-блок.
В выводе компилятора появляется «Результат вычислений в VM: 55».

ШАГ 2 · ЗАПУСК БИНАРНОГО ФАЙЛА

```
./main
```

Выполняются вызовы run().
Начинается раздел 01 о памяти.
Сообщение static повторно не печатается.

Где нужна VM

Нативный бинарник исполняется без VM Nim.
VM также выполняет NimScript (.nims), например сценарии настройки сборки.

ООП: наследование и методы

p03_oop.nim · ТИПЫ И ВЫЗОВ

```nim
type
  Animal = ref object of RootObj
    name: string
  Dog = ref object of Animal
  Cat = ref object of Animal

method speak(a: Animal): string {.base.} =
  "..."
method speak(d: Dog): string = "Woof!"
method speak(c: Cat): string = "Meow!"

let a: Animal = Dog(name: "Rex")
echo a.speak()
```

ВЫВОД ПРОЕКТА

```
Rex говорит: Woof!
Tom говорит: Meow!
Generic говорит: ...
```

method

Выбор реализации по фактическому типу объекта.

proc

Статический выбор перегрузки по типам.

Функциональный стиль

p04_fp.nim

```
import std/[sequtils, sugar]

let numbers = @[1, 2, 3, 4, 5]
let doubled = numbers.map(x => x * 2)
let evens = numbers.filter(x => x mod 2 == 0)
let sum = numbers.foldl(a + b)
```

map	filter	foldl
`@[2, 4, 6, 8, 10]`	`@[2, 4]`	`15`

Все операции используют numbers. let запрещает переприсваивание имени, но не делает любой объект по ссылке глубоко неизменяемым.

Concepts: требования к типу

p05_protocol.nim

```nim
type Comparable = concept a, b
  a < b is bool
  a == b is bool

proc findMax[T: Comparable](
    items: openArray[T]): T =
  if items.len == 0:
    raise newException(ValueError,
      "пустая коллекция")
  result = items[0]
  for item in items:
    if result < item:
      result = item
```

Проверка при компиляции

Тип должен поддерживать операции концепта. Наследование не требуется.

РЕЗУЛЬТАТ FINDMAX

```
ints:    9
strings: cherry
```

Для пустого входа процедура вызывает ValueError.

Шаблоны: подстановка кода

p06_macros.nim · TEMPLATE

```
template debugLog(msg: untyped) =
  echo "[DEBUG] ", msg

debugLog("Это шаблон, он подставляет код.")
```

ВЫВОД ПРОЕКТА

```
[DEBUG] Это шаблон, он подставляет код.
```

Компилятор подставляет тело шаблона в место вызова.
В этом примере echo выполняется при запуске программы.
Гигиена имён снижает риск случайных конфликтов с окружающим кодом.

Макросы: генерация дерева кода

p06_macros.nim · MACRO

```nim
import std/macros

macro repeatPrint(msg: string,
    count: static int): untyped =
  result = newStmtList()
  for i in 0..<count:
    result.add quote do:
      echo `msg`, " (итерация ", `i`, ")"

repeatPrint("Привет от макроса", 3)
```

ВЫВОД

```
Привет от макроса (итерация 0)
Привет от макроса (итерация 1)
Привет от макроса (итерация 2)
```

При компиляции

Макрос создаёт AST с тремя командами echo. count известен заранее.

При запуске

Сгенерированные команды выводят сообщения.

Async / Await: несколько ожиданий

p07_concurrency.nim · БЕЗ ЗАМЕРА ВРЕМЕНИ

```
import std/asyncdispatch

proc simulateRequest(name: string,
    delayMs: int): Future[string] {.async.} =
  await sleepAsync(delayMs)
  result = "Результат от " & name

proc main() {.async.} =
  let f1 = simulateRequest("Сервер A", 100)
  let f2 = simulateRequest("Сервер B", 150)
  let f3 = simulateRequest("Сервер C", 50)
  echo await f1
  echo await f2
  echo await f3

waitFor main()
```

Future[string]

Результат, который станет доступен позже.

await

Приостанавливает текущую процедуру и отдаёт управление диспетчеру.

waitFor

Ведёт цикл событий до готовности Future.

Время ожидания и порядок вывода

Три таймера запускаются до первого await

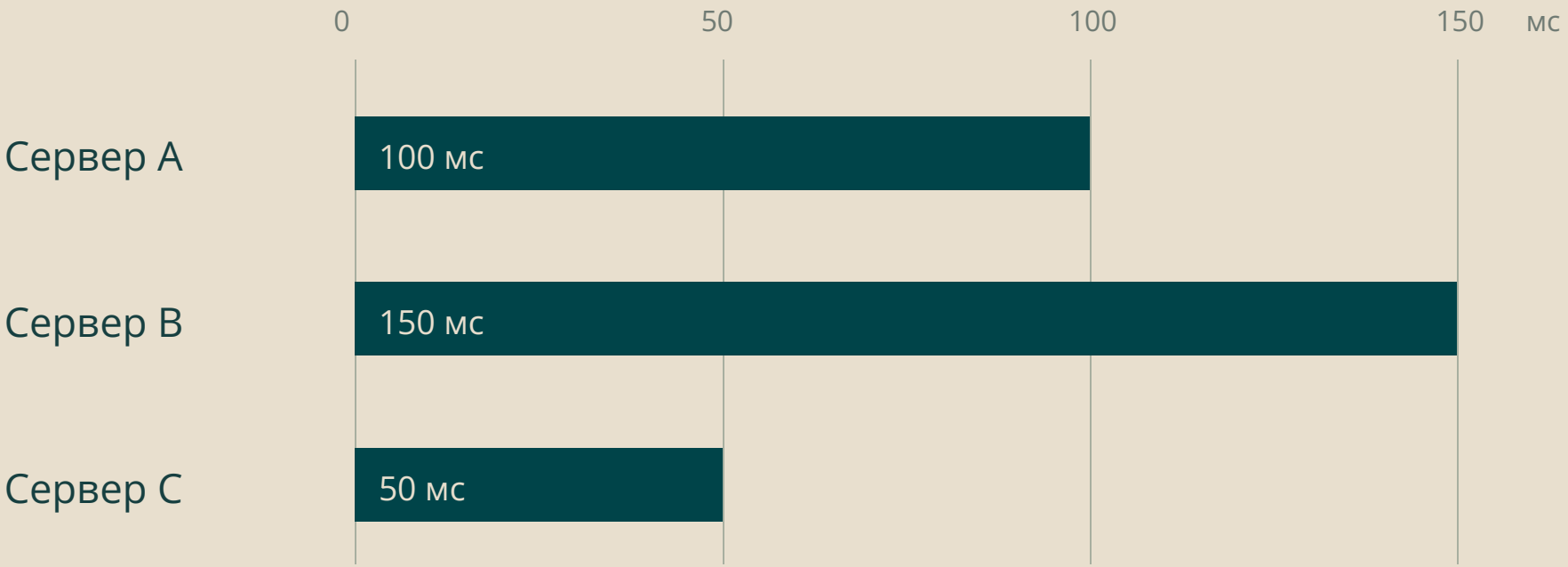


Схема задержек
без накладных расходов

около 0,15 с

вместо суммы
0,10 + 0,15 + 0,05 с

Печать идёт в порядке A, B, C, заданном тремя echo await.
Задача C может быть готова раньше, чем программа выведет её результат.

Пул потоков: spawn и FlowVar

p08_parallelism.nim

```nim
import std/threadpool

proc heavyCalculation(n: int): int64 =
  var sum: int64 = 0
  for i in 1..n:
    sum += int64(i)
  result = sum

let fv1 = spawn heavyCalculation(1_000_000)
let fv2 = spawn heavyCalculation(2_000_000)
sync()
echo ^fv1, ", ", ^fv2
```

spawn

Передаёт вычисление рабочему потоку.

FlowVar[int64]

Хранит будущий результат. Оператор ^ получает значение, при необходимости ожидая его.

sync()

Ждёт завершения задач.

Результаты и общая память

Сумма от 1 до 1 000 000

```
500000500000
```

Сумма от 1 до 2 000 000

```
2000001000000
```

У каждого вызова свой sum

Задачи не изменяют общий счётчик. Для общих изменяемых данных нужны блокировки, атомарные операции или другая согласованная схема доступа.

ORC управляет памятью

Защиту от гонок данных обеспечивает организация доступа между потоками. Ускорение зависит от задачи и затрат на запуск и синхронизацию.

std/threadpool помечен deprecated в Nim 2.2. Для новых проектов документация указывает пакеты malebolgia, taskpools и weave.

Демонстрация проекта

КОМПИЛЯЦИЯ И ЗАПУСК · КАТАЛОГ 03

```
nim c -f -r main.nim
```

ЗАПУСК ЧЕРЕЗ DOCKER

```
docker build -t nim-demo-3 .
docker run --rm nim-demo-3
```

main.nim импортирует восемь модулей.

nim.cfg задаёт --mm:orc и --threads:on.

-f принудительно пересобирает проект.

До запуска

static-блок вычисляет 55.

Разделы 01–06

Деструктор, ООП, map/filter, concept, шаблон и макрос.

Разделы 07–08

Три асинхронных ожидания и две задачи пула потоков.

Nim: итоги трёх презентаций

01 Основы языка

Типы, функции, области видимости и управление потоком

02 Организация программы

Ошибки, модули, обобщения, коллекции и ввод-вывод

03 Продвинутые возможности

ARC / ORC, VM, парадигмы, метапрограммирование и задачи

Спасибо за внимание!

Официальные источники

Память и деструкторы

https://nim-lang.org/docs/mm.html

https://nim-lang.org/docs/destructors.html

Функциональный стиль и макросы

https://nim-lang.org/docs/sequtils.html

https://nim-lang.org/docs/sugar.html

https://nim-lang.org/docs/macros.html

VM компилятора и NimScript

https://nim-lang.org/docs/manual.html

https://nim-lang.org/docs/nims.html

Async / Await

https://nim-lang.org/docs/asyncdispatch.html

ООП и концепты

https://nim-lang.org/docs/tut2.html

https://nim-lang.org/docs/manual_experimental.html

Потоки и синхронизация

https://nim-lang.org/docs/threadpool.html

https://nim-lang.org/docs/locks.html